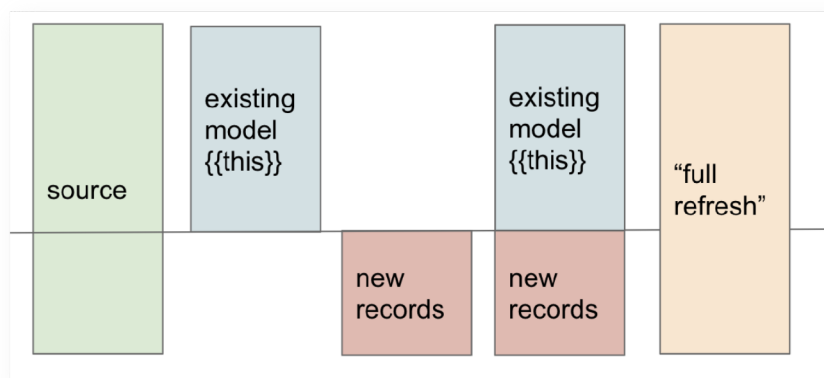


About incremental models

This is an introduction on incremental models, when to use them, and how they work in dbt.

Incremental models in dbt is a [materialization](#) strategy designed to efficiently update your data warehouse tables by only transforming and loading new or changed data since the last run. Instead of processing your entire dataset every time, incremental models append or update only the new rows, significantly reducing the time and resources required for your data transformations.

This page will provide you with a brief overview of incremental models, their importance in data transformations, and the core concepts of incremental materializations in dbt.



A visual representation of how incremental models work. Source: [Materialization best practices guide \(/best-practices/materializations/1-guide-overview\)](#)



LEARN BY VIDEO!

For video tutorials on Incremental models, go to dbt Learn and check out the [Incremental models course](#).

Understand incremental models

Incremental models enable you to significantly reduce the build time by just transforming new records. This is particularly useful for large datasets, where the cost of processing the entire dataset is high.

Incremental models [require extra configuration](#) and are an advanced usage of dbt. We recommend using them when your dbt runs are becoming too slow.

When to use an incremental model

Building models as tables in your data warehouse is often preferred for better query performance. However, using `table` materialization can be computationally intensive, especially when:

- Source data has millions or billions of rows.
- Data transformations on the source data are computationally expensive (take a long time to execute) and complex, like when using Regex or UDFs.

Incremental models offer a balance between complexity and improved performance compared to `view` and `table` materializations and offer better performance of your dbt runs.

In addition to these considerations for incremental models, it's important to understand their limitations and challenges, particularly with large datasets. For more insights into efficient strategies, performance considerations, and the handling of late-arriving data in incremental models, refer to the [On the Limits of Incrementality](#) discourse discussion or to our [Materialization best practices](#) page.

How incremental models work in dbt

dbt's [incremental materialization strategy](#) works differently on different databases. Where supported, a `merge` statement is used to insert new records and update existing records.

On warehouses that do not support `merge` statements, a merge is implemented by first using a `delete` statement to delete records in the target table that are to be updated, and then an `insert` statement.

Transaction management, a process used in certain data platforms, ensures that a set of actions is treated as a single unit of work (or task). If any part of the unit of work fails, dbt will roll back open transactions and restore the database to a good state.

Related docs

- [Incremental models](#) to learn how to configure incremental models in dbt.
- [Incremental strategies](#) to understand how dbt implements incremental models on different databases.
- [Microbatch](#) to understand a new incremental strategy intended for efficient and resilient processing of very large time-series datasets.
- [Materializations best practices](#) to learn about the best practices for using materializations in dbt.

Was this page helpful?

☐ Yes☐ No

[Privacy policy](#) [Create a GitHub issue](#)

This site is protected by reCAPTCHA and the Google [Privacy Policy](#) and [Terms of Service](#) apply.

 [Edit this page](#)

Last updated on **May 4, 2026**